Student Assignment Brief

This document is intended for Coventry University Group students for their own use in completing their assessed work for this module. It must not be passed to third parties or posted on any website. If you require this document in an alternative format, please contact your Module Leader.

Contents:

- Assignment Information
- Assignment Task
- Marking and Feedback
- Assessed Module Learning Outcomes
- Assignment Support and Academic Integrity
- Assessment Marking Criteria

The work you submit for this assignment must be your own independent work, or in the case of a group assignment your own groups' work. More information is available in the 'Assignment Task' section of this assignment brief.

Assignment Information

Module Name: Advanced Algorithms
Module Code: 5022CMD
Assignment Title: Portfolio
Assignment Due: 20th November 2026 (Week 12)
Assignment Credit: 10 credits
Word Count (or equivalent): -
Assignment Type:
Percentage Grade (Applied Core Assessment). You will be provided with an overall grade between 0% and 100%. To pass the assignment you must achieve a grade of 40% or above.

Assignment Task

1. This is an individual assignment. There are **THREE (3)** programs that must be completed and a report to be submitted before the stated due date. The programs will cover a range of data structures and algorithms taught in this module as well as a basic concurrent application.
2. You are required to submit a report as well as upload your complete source code (in Python) for all **THREE (3)** questions to GitHub.
3. Prepare a documentation (in PDF format) for the programming tasks and submit before the due date. The requirements of the report are explained under each question below.
4. There will be a VIVA session for each student to allow you to demonstrate your understanding and highlight the originality of your work. You are required to explain how your code works. The VIVA session will help you to show your fulfilment of the learning outcomes.
5. Read the important notes and marking rubric carefully to avoid any penalties.

Question 1: Hashing

1. Implement a Hash Table that uses **Open Addressing with Linear Probing** as its collision-resolution technique.
2. Develop a local storage system for a pharmacy / medicine retail outlet based on the data structure created in #1. You must create at least **ONE (1)** entity class that can store the products of the pharmacy. Your system must be able to store at least **ONE (1)** type of product from the pharmacy (e.g. medicine). Then, insert a few pre-defined records into the hash table. You can decide the size of the hash table and the number of records to be inserted.
3. Create a command-line Inventory System for the local storage system done in #2. Your Inventory System **must have** an insert function and a search function, while the edit and delete function are optional.
4. **Compare the performance** of searching for records in the hash table with a one-dimensional array. For this comparison to work, you must insert the **same data** into the array and measure the execution time for the searching. Analyse the results and provide explanations of why one is more superior in performance compared with the other.
5. Provide the following discussions in your report:

i. Screenshot the code segment which defines the structure of the slots/buckets of the Hash Table. Explain your choice (e.g. simplicity, efficiency, etc.) of the structure used for collision resolution and probing.

ii. Screenshot the code segment which defines the entity class of a pharmacy product. Explain the choice of the class attributes and its data types.

iii. Provide a complete documentation on the experiment done in #4 above. Your documentation must include sample code screenshots, output screenshots, your analysis, and a conclusion.

Sample Expected Output (for guidance only — your actual values, formatting, and slot numbers will differ):

```
==== Pharmacy Inventory System ====
1. Insert Product
2. Search Product
3. Exit
Enter choice: 1
Enter Product ID: 1042
Enter Product Name: Paracetamol 500mg
Enter Price: 5.90
Enter Stock Quantity: 120
Product inserted successfully into slot [42]

Enter choice: 2
Enter Product ID to search: 1042
Product Found!
ID: 1042 | Name: Paracetamol 500mg | Price: RM5.90 | Stock: 120
Search completed in 0.000031 seconds (Hash Table)
Search completed in 0.002184 seconds (Array - same record)

Enter choice: 3
Exiting Inventory System. Goodbye!
```

Question 2: Graph

1. Construct an unweighted directed graph data structure. Include the following methods in your graph data structure:
   - **addVertex**: add a new vertex to the graph
   - **addEdge**: connect one vertex with another vertex
   - **listIncomingAdjacentVertex:** for a given vertex, this method should list all vertices from which an edge is incoming into this vertex

   > **Programming Best Practice (Abstraction):** It is recommended to keep your graph data structure free from any code related to a specific domain/entity (i.e. Course entity). Keep the data type generic for the vertices in your Graph data structure to make your Graph data structure reusable in other contexts. Keeping your code reusable is a good programming practice.

2. Create a Course domain / entity class that represents a single course offered by a university. You may select any relevant attributes for your Course entity class. Examples of attributes are courseCode, courseName, creditHours, level (core / elective), etc.
3. Create the profile for a **minimum of FIVE (5)** courses by creating Course objects. Limit the number of courses to a **maximum of TEN (10).**
4. Create a **graph object** that holds the data for a simplified course-prerequisite system. The graph must be able to hold the data of the prerequisites and the courses unlocked by every course. You may design the connections in any way, in terms of which course requires which other course(s) as a prerequisite. Mimic a university registration system such that a course can require another course as a prerequisite, and can also be required by other courses in return, but not necessarily.
5. Design a menu-driven program that allows the user to perform the following actions:

   ➢ **Mandatory Features (Compulsory):**

   a. Display a list of all the courses' names.
   b. View the details of any one course in detail (Ignore the level/visibility setting of the course and display all information).
   c. View the list of prerequisite courses required by a particular course (This involves listing all the incoming edges of a particular vertex).

d. View the list of courses that a particular course unlocks, i.e. the courses for which it is a prerequisite (You may implement this in any way workable and efficient).

- **Advanced Features (Optional. You may select any one or combinations to implement):**

a. Add a course on-demand (This involves creating a new vertex on-demand in the graph).

b. For courses marked as elective, their profile only shows the course name, and the other details are hidden. For courses marked as core, their profile shows the name and all other details (This involves showing the details of a single vertex object).

c. Allow a course X to **add** another course Y as a prerequisite on-demand (This involves creating a new incoming edge from Y to X on-demand).

d. Allow a course X to **remove** another course Y as a prerequisite on-demand (This involves removing an edge from Y to X on-demand).

If you wish to develop a GUI (Graphical User Interface) instead of a command-line interface (CLI), you may do so, but it will **not result in additional marks** as it is not under the learning outcomes.

6. Provide the following discussions in your report.

1. Screenshot the code segments that handle the mandatory features and provide brief explanations on how the code works.
2. [For those who had done any of the extra features] Screenshot the code segments that handle the extra features that you implemented and provide brief explanations on how the code works.
3. Provide screenshots for the output for all the features implemented. Use test data to demonstrate that your program is working properly. Include brief explanations whenever necessary if they can clarify your points.

Sample Expected Output (for guidance only — your actual course list and codes will differ):

```
==== University Course Prerequisite System ====
1. View All Courses
2. View Course Details
3. View Prerequisites of a Course
4. View Courses Unlocked by a Course
5. Exit
Enter choice: 1

Available Courses:
1. CS101 - Introduction to Programming
2. CS201 - Data Structures
3. CS301 - Algorithms
4. CS302 - Database Systems
5. CS401 - Software Engineering

Enter choice: 3
Enter Course Code: CS301
Prerequisites for CS301 - Algorithms:
- CS201 (Data Structures)
- CS101 (Introduction to Programming)

Enter choice: 4
Enter Course Code: CS201
Courses unlocked by CS201 - Data Structures:
- CS301 (Algorithms)
- CS302 (Database Systems)

Enter choice: 5
Exiting program. Goodbye!
```

Question 3: Concurrent process

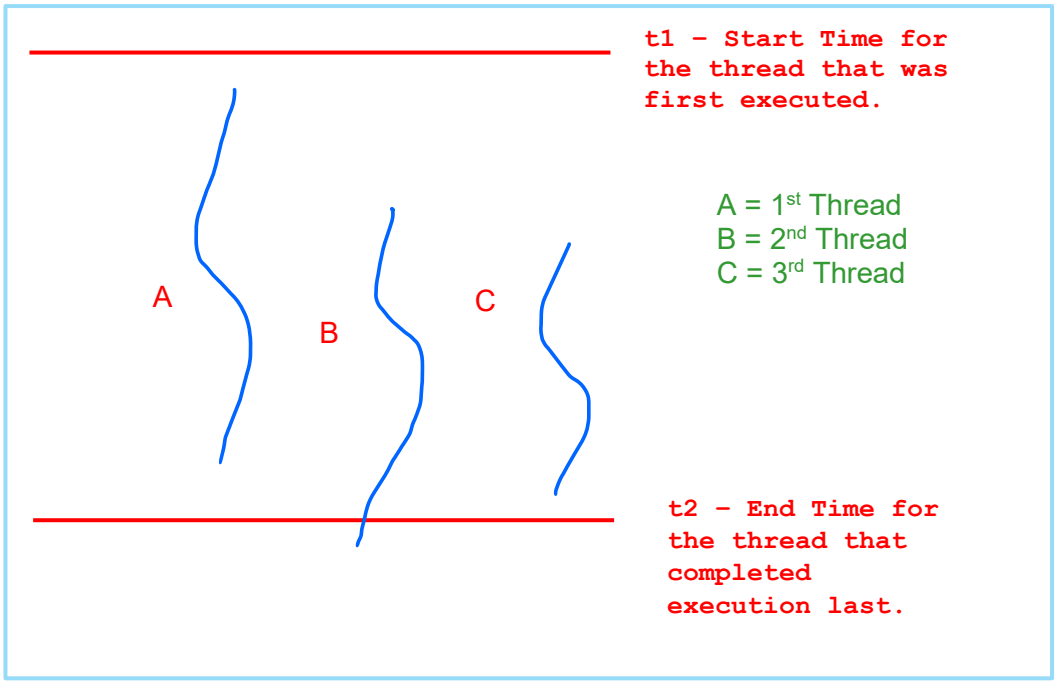
1. Python programs support multithreading. Discuss whether, in Python, this is concurrent processing or parallel processing.
2. Write a function that calculates the nth Fibonacci number based on a given number. Derive the Big-O of this function by counting the number of primitive operations and explain its time complexity.
3. Using multithreading:
    - Write a program that calculates Fibonacci(35), Fibonacci(38), and Fibonacci(40). Use multithreading and create a separate thread to handle each calculation (i.e. you must create 3 separate threads, 1 thread for each value).
    - Measure the time taken (in nanoseconds) to complete the operation of generating the three Fibonacci numbers using multithreading. The total time can be calculated with the following formula:

        Time_Elapsed = End_Time_Of_Thread_Finished_Last – Start_Time_Of_Thread_That_Started_First

Learn how to get the current system time in nanoseconds for Python from the link below:

```
https://docs.python.org/3/library/time.html
```

The illustration below shows how to calculate the total time taken for the 3 sets of operation in separate threads.



$T = t_2 - t_1$

- Perform testing in 10 rounds for generating the same numbers and then observe the time taken (T) for each round as well as the average time taken (Average for T) for all 10 rounds. Your program should show the T value in nanoseconds for each round of operation. Format the output as desired but make the output clear and easy to read.

4. Perform the same operations as #3 (i.e. generate Fibonacci(35), Fibonacci(38), and Fibonacci(40)), this time **without multithreading**. Calculate the total time taken for each round and the average time taken and display the results in your program output.
5. Provide the following discussions in your report:
   i. Provide a screenshot of the code for the function that calculates the Fibonacci number. Derive the Big-O of the function and explain its time complexity.
   ii. Based on the experiments that you had done in Step #3 and Step #4 above, make a comparison and provide an analysis of your results and findings. Explain whether multithreading shortens the time taken for the tasks. You must include screenshots of the output with some sample data. Your screenshots must show the total and average time taken for each experiment.
   iii. Screenshot the code segment that handles multithreading and explain how the code works.

iv. Discuss why multithreading did or did not shorten the time taken for this experiment and suggest an example of a situation where multithreading will improve the performance.

Sample Expected Output (for guidance only — your actual timing values will differ depending on your machine):

```
=== Fibonacci Computation (With Multithreading) ===
Round 1 : Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 1,532,847,210 ns
Round 2 : Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 1,498,032,556 ns
Round 3 : Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 1,521,774,902 ns
...
Round 10: Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 1,511,209,873 ns
Average T (With Multithreading)    : 1,509,664,212 ns

=== Fibonacci Computation (Without Multithreading) ===
Round 1 : Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 3,021,553,981 ns
Round 2 : Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 3,008,221,447 ns
...
Round 10: Fib(35)=9227465, Fib(38)=39088169, Fib(40)=102334155 | T = 2,996,884,310 ns
Average T (Without Multithreading) : 3,012,447,665 ns

Observation: Multithreading reduced the average completion time by approximately 49.9%.
```

Challenges Faced and Personal Reflections in Doing This Assignment

In no more than 2 pages, write about your personal experience while doing this assignment. You may share about the challenges faced, overcoming obstacles, solving problems, picking up new skills, managing time, developing new habits, cultivating discipline, etc.

You may share anything that is sharable and express your feelings about going through the process of completing this assignment.

Marking and Feedback

How will my assignment be marked?

Your assignment will be marked by the module leader.

How will I receive my grades and feedback?

Provisional marks will be released once internally moderated.

Feedback will be provided by the module leader alongside grades release.

Marks and feedback can be accessed by going to the Grades on Canvas system.

Your provisional marks and feedback should be available by end of the semester.

What will I be marked against?

Details of the marking criteria for this task can be found at the bottom of this assignment brief.

Assessed Module Learning Outcomes

The Learning Outcomes for this module align to the marking criteria which can be found at the end of this brief. Ensure you understand the marking criteria to ensure successful achievement of the assessment task.

The following module learning outcomes are assessed in this task:

1. Evaluate the complexity and efficiency of algorithms for solving a range of problems
2. Apply algorithms and data structures to solve novel problems
3. Demonstrate solutions to intractable problems by considering computational complexity and algorithm theory.
4. Analyse the issue of data consistency and multi-threading in a concurrent application.

Assignment Support and Academic Integrity

Spelling, Punctuation, and Grammar:

You are expected to use effective, accurate, and appropriate language within this assessment task.

Academic Integrity:

The work you submit must be your own, or in the case of groupwork, that of your group. All sources of information need to be acknowledged and attributed; therefore, you must provide references for all sources of information and acknowledge any tools used in the production of your work. We use detection software and make routine checks for evidence of academic misconduct.

It is your responsibility to keep a record of how your thinking has developed as you progress through to submission. Appropriate evidence could include version-controlled documents, developmental sketchbooks, or journals. This evidence can be called upon if we suspect academic misconduct.

If using Artificial Intelligence (AI) tools in the development of your assignment, you must reference which tools you have used and for what purposes you have used them. This information must be acknowledged in your final submission.

Unable to Submit on Time?

Student will be graded as ZERO if they are unable to submit on time.

Administration of Assessment

Module Leader Name: Vimala Doraisamy
Module Leader Email: vimala.doraisamy@newinti.edu.my
Assignment Category: Portfolio
Attempt Type: [Standard / Resit / Deferral]
Component Code: CW

Assessment Marking Criteria

Criteria	Weightage
Q1 Code	10%
Q2 Code	10%
Q3 Code	10%
Q1 Report Writing	10%
Q2 Report Writing	10%
Q3 Report Writing	10%
VIVA	40%
TOTAL	**100%**

Detailed Assessment Criteria

	Fail 0–29%	Fail 30–39%	40–49% [Basic]	50–59% [Average]	60–69% [Good]	70–79% [Excellent]	80–100% [Outstanding]
Q1 Code (Completeness & Error-Free) Weighting: 10%	This part is missing in the report, OR the GitHub link is inaccessible or not provided, OR code is missing.	GitHub link provided and accessible, but the code failed to run or compiled with errors, OR failed to fulfil all basic requirements.	Code performs all basic tasks. No output, or output is minimal/lacks crucial information. No abstraction applied; a lot of hardcoding.	Code performs all basic tasks. Output shows required information with very basic or no formatting. No abstraction; some hardcoding.	Code performs more than the basic tasks. Output well formatted. Some abstraction applied; parts of the code reusable. No unnecessary hardcoding.	Code performs more than the basic tasks. Output well formatted with good abstraction; most of the data structure code reusable.	Clean, well-commented, well-indented code performing more than the basic tasks. Excellent formatting, excellent abstraction, fully reusable, no unnecessary hardcoding.
Q2 Code (Completeness & Error-Free) Weighting: 10%	This part is missing in the report, OR the GitHub link is inaccessible or not provided, OR code is missing.	GitHub link provided and accessible, but the code failed to run or compiled with errors, OR failed to fulfil all basic requirements.	Code performs all basic tasks. No output, or output is minimal/lacks crucial information. No abstraction applied; a lot of hardcoding.	Code performs all basic tasks. Output shows required information with very basic or no formatting. No abstraction; some hardcoding.	Code performs more than the basic tasks. Output well formatted. Some abstraction applied; parts of the code reusable. No unnecessary hardcoding.	Code performs more than the basic tasks. Output well formatted with good abstraction; most of the data structure code reusable.	Clean, well-commented, well-indented code performing more than the basic tasks. Excellent formatting, excellent abstraction, fully reusable, no unnecessary hardcoding.
Q3 Code (Completeness & Error-Free) Weighting: 10%	This part is missing in the report, OR the GitHub link is inaccessible or not provided, OR code is missing.	GitHub link provided and accessible, but the code failed to run or compiled with errors, OR failed to fulfil all basic requirements.	Code performs all basic tasks. No output, or output is minimal/lacks crucial information. No abstraction applied; a lot of hardcoding.	Code performs all basic tasks. Output shows required information with very basic or no formatting. No abstraction; some hardcoding.	Code performs more than the basic tasks. Output well formatted. Some abstraction applied; parts of the code reusable. No unnecessary hardcoding.	Code performs more than the basic tasks. Output well formatted with good abstraction; most of the data structure code reusable.	Clean, well-commented, well-indented code performing more than the basic tasks. Excellent formatting, excellent abstraction, fully reusable, no unnecessary hardcoding.
Q1 Writing Weighting: 10%	This part is missing, or the work submitted is inadequate and incomplete.	Missing code explanation or output screenshots. Discussions are rudimentary and below the acceptable level.	Code explanation merely narrates the code rather than explaining it. Discussions/output lack clarity, detail, or miss crucial points.	Able to explain the code with average breadth and depth. Shows output clearly with a basic level of discussion with adequate coverage.	Able to explain the code with average breadth and depth. Shows output clearly with an average level of discussion with good coverage.	Able to explain the code with proper breadth but lacking depth; adequate coverage. Shows output clearly with good discussion and excellent coverage.	Able to explain the code with proper depth and breadth, not over-explained. Shows output clearly with high-quality discussion.
Q2 Writing Weighting: 10%	This part is missing, or the work submitted is	Missing code explanation or output screenshots. Discussions are	Code explanation merely narrates the code rather than explaining it.	Able to explain the code with average breadth and depth. Shows output	Able to explain the code with average breadth and depth. Shows output	Able to explain the code with proper breadth but lacking depth; adequate	Able to explain the code with proper depth and breadth, not over-explained.

	inadequate and incomplete.	rudimentary and below the acceptable level.	Discussions/output lack clarity, detail, or miss crucial points.	clearly with a basic level of discussion with adequate coverage.	clearly with an average level of discussion with good coverage.	coverage. Shows output clearly with good discussion and excellent coverage.	Shows output clearly with high-quality discussion.
Q3 Writing Weighting: 10%	This part is missing, or the work submitted is inadequate and incomplete.	Missing code explanation or output screenshots. Discussions are rudimentary and below the acceptable level.	Code explanation merely narrates the code rather than explaining it. Discussions/output lack clarity, detail, or miss crucial points.	Able to explain the code with average breadth and depth. Shows output clearly with a basic level of discussion with adequate coverage.	Able to explain the code with average breadth and depth. Shows output clearly with an average level of discussion with good coverage.	Able to explain the code with proper breadth but lacking depth; adequate coverage. Shows output clearly with good discussion and excellent coverage.	Able to explain the code with proper depth and breadth, not over-explained. Shows output clearly with high-quality discussion.
VIVA — Mastery over own work Weighting: 20%	Failed to turn up for the VIVA, or turned up too late to demonstrate this ability.	Failed to explain the code submitted even for the basic things.	Only able to explain very basic things about the code; unable to explain more complex parts.	Able to explain some of the code submitted but showed uncertainties in some parts.	Able to explain most of the code with moderate clarity and confidence; some minor inaccuracies.	Able to explain most of the code with high clarity and confidence; some minor inaccuracies.	Able to explain all the code submitted with high clarity, confidence, and accuracy.
VIVA — Solving new situations Weighting: 10%	Failed to turn up for the VIVA, or failed to provide a solution for a new problem.	Failed to provide a solution to the new problem, but gave a rough idea close to a possible solution.	Able to provide an average solution but unable to express it with further detail.	Able to provide an average solution and able to express it with adequate detail.	Able to provide a good solution but unable to express the idea with adequate detail.	Moderately articulate in expressing the solution, demonstrating good problem-solving skills.	Very articulate in expressing the solution, demonstrating excellent out-of-the-box thinking and novel solutions.
VIVA — Evidence of basic skills Weighting: 10%	Failed to turn up for the VIVA, or turned up too late to demonstrate this ability.	Failed to demonstrate even basic skills required; showed very weak understanding.	Showed a lot of signs of uncertainty in carrying out the basic tasks.	Showed some signs of uncertainty in carrying out the basic tasks.	Showed confidence and some proficiency in carrying out the basic tasks.	Showed confidence and high proficiency in carrying out the tasks.	Demonstrated outstanding skills in programming, higher than expected at college level.

Note: Submitting a long report with many pages or many paragraphs does not guarantee high marks. The conciseness, relevance, accuracy, depth, coverage and quality are several of the qualities assessed in your report. Focus on quality, not the number of words or pages. Furthermore, during the VIVA you will be asked about selected parts of your report.